



Arquitetura Serverless para API REST de Gestão de Pedidos

Grupo 3 Engenharia de cloud:

Ítalo Calábria, Lucas Satlher, Diego Lima, Jhonatas Dias, William Pizziolo



Contexto e problema

Cenário

Uma empresa de comércio digital busca modernizar sua arquitetura substituindo uma aplicação tradicional por uma solução 100% *serverless* em nuvem para o gerenciamento de pedidos. O objetivo do projeto é desenvolver uma API REST escalável, de baixo custo operacional e com alta disponibilidade, capaz de realizar o cadastro, consulta, atualização, cancelamento e listagem de pedidos em um banco de dados NoSQL gerenciado.

Desafios:

- 1- Escalabilidade para atender aumento de demanda.
- 2- Redução de custos de infraestrutura.
- 3- Monitoramento de falhas e desempenho.
- 4- Facilidade de manutenção.

Solução proposta

Implementar uma arquitetura Serverless utilizando AWS.

Arquitetura da solução

ARQUITETURA SERVERLESS DE PEDIDOS

1. CLIENTE / POSTMAN

Inicia o fluxo enviando uma requisição HTTP para o endpoint da aplicação.

2. API GATEWAY

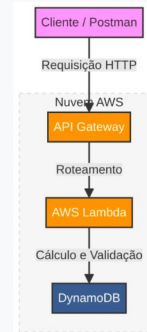
Gerencia o roteamento e a segurança da requisição de entrada dentro da AWS.

3. AWS LAMBDA

Calcula o valor total e valida o status do pedido com lógica sob demanda.

4. DYNAMODB

Armazenamento NoSQL de alta performance para persistência dos dados.



Serviços utilizados



API Gateway

Build, Deploy and Manage APIs



CloudWatch

Monitor Resources and Applications



Lambda

Run code without thinking about servers



DynamoDB

Managed NoSQL Database

Preparação do ambiente

Definição da região

CloudShell

us-east-1



```
~ $ aws --version
aws-cli/2.34.64 Python/3.14.5 Linux/6.1.172-216.329.amzn2023.x86_64 exec-env/CloudShell exe/x86_64.amzn.2023
~ $ aws sts get-caller-identity
{
  "UserId": "606114585658",
  "Account": "606114585658",
  "Arn": "arn:aws:iam::606114585658:root"
}
~ $ aws configure get region
sa-east-1
~ $ export AWS_REGION=us-east-1
~ $ aws configure set region $AWS_REGION
~ $ echo "Região configurada: $AWS_REGION"
Região configurada: us-east-1
~ $
```

Definição das variáveis globais

CloudShell

us-east-1



```
~ $ export TABLE_NAME="pedidos"
~ $ echo $TABLE_NAME
pedidos
~ $ export ROLE_NAME="role-lambda-pedidos"
~ $ echo $ROLE_NAME
role-lambda-pedidos
~ $ export POLICY_NAME="policy-lambda-pedidos"
~ $ echo $POLICY_NAME
policy-lambda-pedidos
~ $ export LAMBDA_NAME="gestao-pedidos-lambda"
~ $ echo $LAMBDA_NAME
gestao-pedidos-lambda
~ $ export API_NAME="api-pedidos-serverless"
~ $ echo $API_NAME
api-pedidos-serverless
~ $ export ACCOUNT_ID=$(aws sts get-caller-identity --query Account --output text)
~ $ echo $ACCOUNT_ID
606114585658
~ $ export TABLE_ARN="arn:aws:dynamodb:$AWS_REGION:$ACCOUNT_ID:table/$TABLE_NAME"
~ $ echo $TABLE_ARN
arn:aws:dynamodb:us-east-1:606114585658:table/pedidos
~ $ export LAMBDA_ARN=$(aws lambda get-function --function-name "$LAMBDA_NAME" --region "$AWS_REGION" --query "Configuration.FunctionArn" --output text)
~ $ echo $LAMBDA_ARN
arn:aws:lambda:us-east-1:606114585658:function:gestao-pedidos-lambda
~ $
```

Criação da role Lambda

CloudShell

us-east-1



```
~ $ cat > trust-policy.json <<'EOF'
> {
>   "Version": "2012-10-17",
>   "Statement": [
>     {
>       "Effect": "Allow",
>       "Principal": {
>         "Service": "lambda.amazonaws.com"
>       },
>       "Action": "sts:AssumeRole"
>     }
>   ]
> }
> EOF
~ $ aws iam create-role --role-name "$ROLE_NAME" --assume-role-policy-document file://trust-policy.json
{
  "Role": {
    "Path": "/",
    "RoleName": "role-lambda-pedidos",
    "RoleId": "AROAY2HZ6TQ5OL2N07RJL",
    "Arn": "arn:aws:iam::606114585658:role/role-lambda-pedidos",
    "CreateDate": "2026-06-21T14:20:25+00:00",
    "AssumeRolePolicyDocument": {
      "Version": "2012-10-17",
      "Statement": [
        {
          "Effect": "Allow",
          "Principal": {
            "Service": "lambda.amazonaws.com"
          },
          "Action": "sts:AssumeRole"
        }
      ]
    }
  }
}
~ $ export ROLE_ARN=$(aws iam get-role --role-name "$ROLE_NAME" --query "Role.Arn" --output text)
~ $ echo $ROLE_ARN
arn:aws:iam::606114585658:role/role-lambda-pedidos
~ $
```

Políticas DynamoDB e CloudWatch

CloudShell

us-east-1



```
~ $ cat > lambda-policy.json <<EOF
~ {
~   "Version": "2012-10-17",
~   "Statement": [
~     {
~       "Sid": "DynamoDBPedidosAccess",
~       "Effect": "Allow",
~       "Action": [
~         "dynamodb:PutItem",
~         "dynamodb:GetItem",
~         "dynamodb:Scan",
~         "dynamodb:UpdateItem",
~         "dynamodb>DeleteItem"
~       ],
~       "Resource": "$TABLE_ARN"
~     },
~     {
~       "Sid": "CloudWatchLogsAccess",
~       "Effect": "Allow",
~       "Action": [
~         "logs:CreateLogGroup",
~         "logs:CreateLogStream",
~         "logs:PutLogEvents"
~       ],
~       "Resource": "*"
~     }
~   ]
~ }
~ EOF
~ $ aws iam put-role-policy --role-name "$ROLE_NAME" --policy-name "$POLICY_NAME" --policy-document file://lambda-policy.json
~ $
```


Criação da API Gateway

CloudShell

us-east-1



```
~ $ aws apigatewayv2 create-api --name "$API_NAME" --protocol-type HTTP --target "$LAMBDA_ARN" --region "$AWS_REGION" > api.json
~ $ cat api.json
{
  "ApiEndpoint": "https://1eapm3cb05.execute-api.us-east-1.amazonaws.com",
  "ApiId": "1eapm3cb05",
  "ApiKeySelectionExpression": "$request.header.x-api-key",
  "CreateDate": "2026-06-22T02:48:55+00:00",
  "DisableExecuteApiEndpoint": false,
  "IpAddressType": "ipv4",
  "Name": "api-pedidos-serverless",
  "ProtocolType": "HTTP",
  "RouteSelectionExpression": "$request.method $request.path"
}
~ $ export API_ID=$(cat api.json | jq -r ".ApiId")
~ $ export API_ENDPOINT=$(cat api.json | jq -r ".ApiEndpoint")
~ $ echo $API_ID
1eapm3cb05
~ $ echo $API_ENDPOINT
https://1eapm3cb05.execute-api.us-east-1.amazonaws.com
~ $
```

lambda invoker

CloudShell

Ações ▼

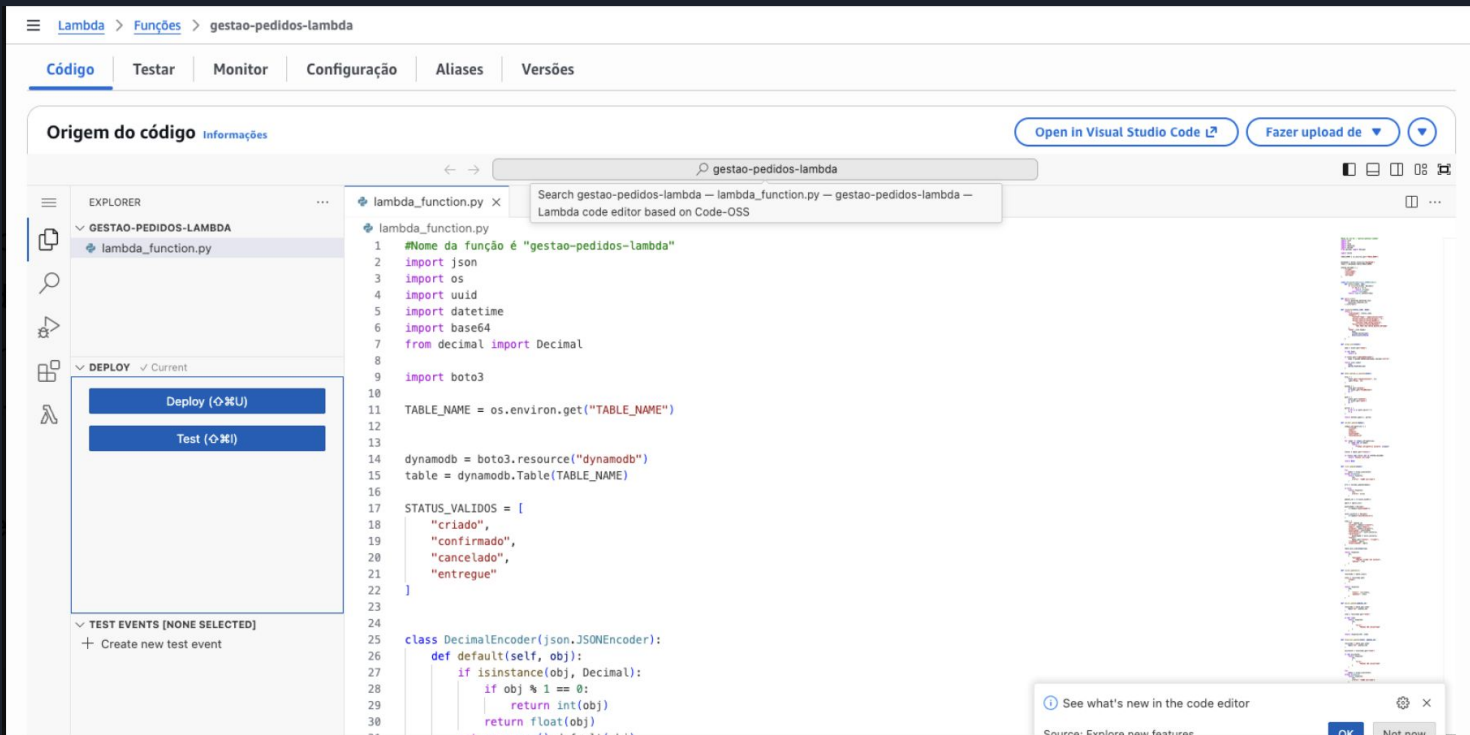


us-east-1



```
~ $ aws lambda add-permission --function-name "$LAMBDA_NAME" --statement-id "apigateway-invoke" --action lambda:InvokeFunction --principal apigateway.amazonaws.com --source-arn "arn:aws:executeapi:$AWS_REGION:$ACCOUNT_ID:$API_ID/*/*" --region "$AWS_REGION"
{
  "Statement": "[{"Sid": "apigateway-invoke", "Effect": "Allow", "Principal": {"Service": "apigateway.amazonaws.com"}, "Action": "lambda:InvokeFunction", "Resource": "arn:aws:lambda:us-east-1:606114585658:function:gestao-pedidos-lambda", "Condition": {"ArnLike": {"AWS:SourceArn": "arn:aws:executeapi:us-east-1:606114585658:leapn3cb85/*/*/*"}}}]"
}
~ $
```

Função Lambda



The screenshot displays the AWS Lambda console interface for a function named "gestao-pedidos-lambda". The top navigation bar includes tabs for "Código", "Testar", "Monitor", "Configuração", "Aliases", and "Versões". The "Código" tab is active, showing the "Origem do código" section with a search bar and buttons for "Open in Visual Studio Code" and "Fazer upload de".

The left sidebar contains the "EXPLORER" view, showing the file structure of the function. The "lambda_function.py" file is selected, and its content is displayed in the main editor area. The code is a Python script that defines a lambda function named "gestao-pedidos-lambda". It imports necessary modules like "json", "os", "uuid", "datetime", "base64", "Decimal", and "boto3". It also defines a "DecimalEncoder" class to handle decimal values in JSON.

The right sidebar shows the "DEPLOY" section, which includes buttons for "Deploy" and "Test". Below this, there is a section for "TEST EVENTS" with a "Create new test event" button.

```
1 #Nome da função é "gestao-pedidos-lambda"
2 import json
3 import os
4 import uuid
5 import datetime
6 import base64
7 from decimal import Decimal
8
9 import boto3
10
11 TABLE_NAME = os.environ.get("TABLE_NAME")
12
13
14 dynamodb = boto3.resource("dynamodb")
15 table = dynamodb.Table(TABLE_NAME)
16
17 STATUS_VALIDOS = [
18     "criado",
19     "confirmado",
20     "cancelado",
21     "entregue"
22 ]
23
24
25 class DecimalEncoder(json.JSONEncoder):
26     def default(self, obj):
27         if isinstance(obj, Decimal):
28             if obj % 1 == 0:
29                 return int(obj)
30             return float(obj)
31         return super().default(obj)
```

Função Lambda 1.2 - Variável "TABLE_NAME"

☰ [Lambda](#) > [Funções](#) > gestao-pedidos-lambda

🟢 A função "gestao-pedidos-lambda" foi atualizada com êxito. ✕

Configuração

- Configuração geral
- Gatilhos
- Permissões
- Destinos
- URL da função
- Variáveis de ambiente**
- Tags
- VPC
- Bancos de dados do RDS
- Ferramentas de monitoramento e operações
- Deteção de recursão e simultaneidade
- Invocação assíncrona
- Assinatura de código
- Sistemas de arquivos
- Máquinas de estado

Variáveis de ambiente (1) Edit

As variáveis de ambiente a seguir são criptografadas em repouso com a chave de serviço padrão do Lambda.

🔍 Search Variáveis de ambiente

Chave	Valor
TABLE_NAME	pedidos

Tabela DynamoDB

pedidos

Última atualização
June 23, 2026, 20:49 (UTC-3:00)

⌂ ☆ Ações Explorar itens da tabela

Configurações Índices Monitorar Tabelas globais Backups Exportações e streams Permissões

Proteja sua tabela do DynamoDB contra gravações e exclusões acidentais

Quando você ativa a recuperação para um ponto no tempo (PITR), o DynamoDB faz backup dos dados da tabela automaticamente para que você possa restaurar a qualquer segundo nos últimos 1 a 35 dias. Cobranças adicionais se aplicam. [Saiba mais](#)

Editar PITR

Informações gerais Informações

Chave de partição
Id (String)

Alarmes
Nenhum alarme ativo

Tamanho médio do item
200 bytes

Nome do recurso da Amazon (ARN)
 arn:aws:dynamodb:us-east-1:606114585658:table/pedidos

Chave de classificação
-

Recuperação em um ponto anterior no tempo (PITR) Informações
Desativado

Política baseada em recursos Informações
Não ativo

Modo de capacidade
Sob demanda

Contagem de itens
12

Status da tabela
Ativo

Tamanho da tabela
2.4 quilobytes

Obter contagem de itens ativos

Informações adicionais

Classe da tabela
DynamoDB Standard

Stream do DynamoDB
Desativado

Proteção contra exclusão
Desativado

Índices
0 globais, 0 locais

Tempo de vida (TTL) Informações
Desativado

Integrações Informações
0

Regiões de replicação
0

Criptografia
Chave de propriedade da AWS

Consistência multirregional
-

Data de criação
junho 21, 2026, 16:24:34 (UTC-03:00)

Endpoints Implementados

API Gateway > APIs > api-pedidos-serverless (9ec1av5s88) > Integrações

API Gateway

APIs

Nomes de domínio personalizados

Associações de acesso a nomes de domínio

Links de VPC

API: api-pedidos-ser...(9ec1av5s88)

Develop

Routes

Authorization

Integrations

CORS

Reimport

Export

Deploy

Stages

Monitor

Metrics

Logging

Protect

Throttling

Integrações

Estágio: - Implantar

Anexar integrações a rotas Gerenciar integrações

Rotas para api-pedidos-serverless

Q Pesquisar

ANY

▼ /pedidos

POST AWS Lambda

GET AWS Lambda

▼ /{id}

PUT AWS Lambda

DELETE AWS Lambda

GET AWS Lambda

▼ /status

PATCH AWS Lambda

Detalhes da integração para a rota

PATCH /pedidos/{id}/status (kknorxl)

Função do Lambda
gestao-pedidos-lambda (us-east-1) [🔗](#)

ID de integração
iuqeubr

Descrição
Atualiza parte dos dados de um pedido.

Versão do formato da carga
The parsing algorithm for the payload sent to and returned from your Lambda function. [Learn more](#)
2.0 (formato de resposta interpretado)

Invocar permissões
A política de recursos da função do Lambda determina se o API Gateway pode invocá-la. Você pode executar o trecho de comando da AWS CLI abaixo para conceder ao API Gateway permissão para invocar a função do AWS Lambda.

Exemplo de instrução de política

Tempo limite
O número de milissegundos que o API Gateway deve aguardar por uma resposta da integração antes de atingir o tempo limite.
30000

Mapeamento de parâmetros de solicitação
Não configurado

Mapeamentos de parâmetros de resposta
Não configurado

Logs CloudWatch

/aws/lambda/gestao-pedidos-lambda

Ações

Visualizar no Logs Insights

Começar a seguir

Pesquisar grupo de logs

▼ Detalhes do grupo de logs

Classe de log

Informações

Padrão

ARN

arn:aws:logs:us-east-1:606114585658:log-group:/aws/lambda/gestao-pedidos-lambda:

Hora de criação

2 dias atrás

Retenção

Nunca expirar

Bytes armazenados

29.18 KB

Filtros de métrica

0

Filtros de assinatura

0

Regras do Contributor Insights

-

ID da chave do KMS

-

Proteção contra exclusão

Desligado

Proteção de dados

-

Contagem de dados confidenciais

-

Índices de campo personalizados

Configurar

Transformador

Configurar

Deteção de anomalias

Configurar

Autenticação por token do portador

Desligado

Ingestão de syslog

Informações

Configurar

Streams de log (13)

Excluir

Criar stream de logs

Pesquisar todos os fluxos de log

Por padrão, carregamos apenas os fluxos de log mais recentes.

Q

Filtrar streams de log ou tentar pesquisa de prefixo

Correspondência exata

Mostrar expirados

Informações

<

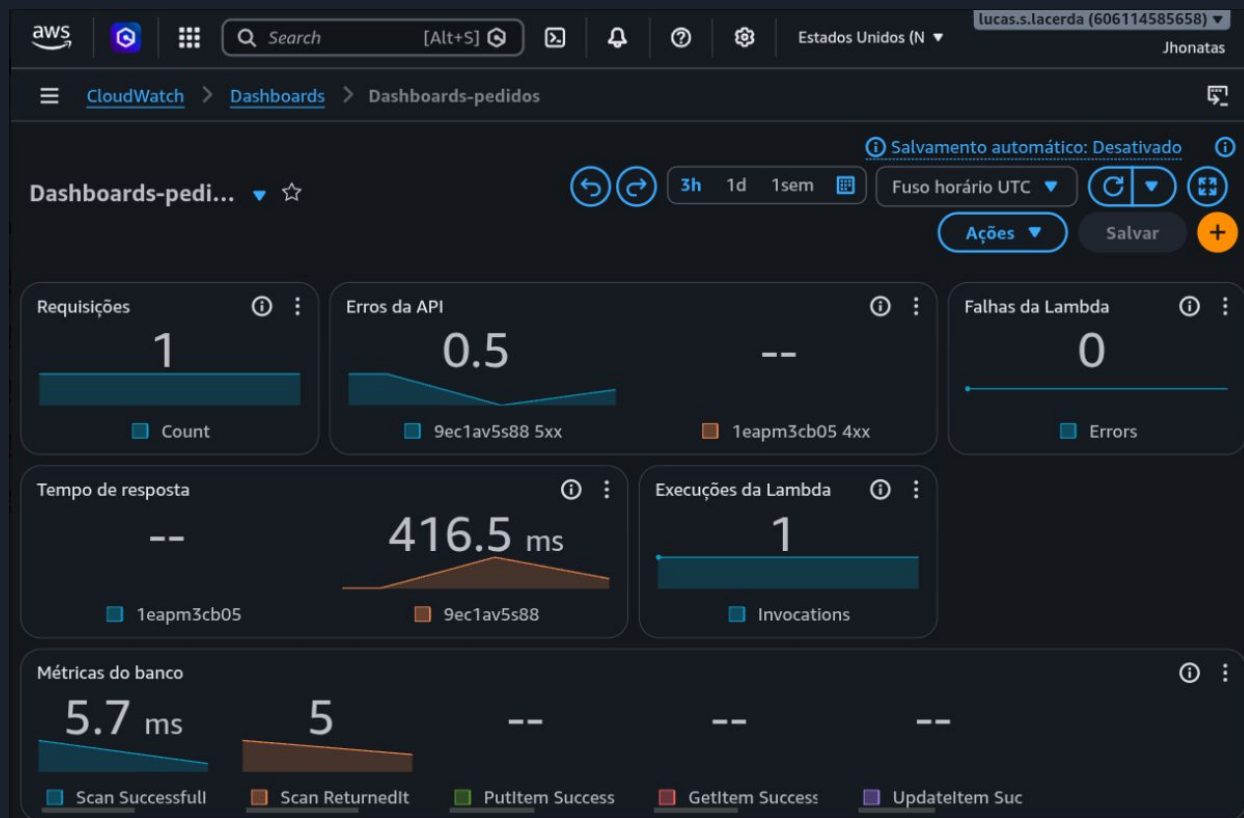
1

>

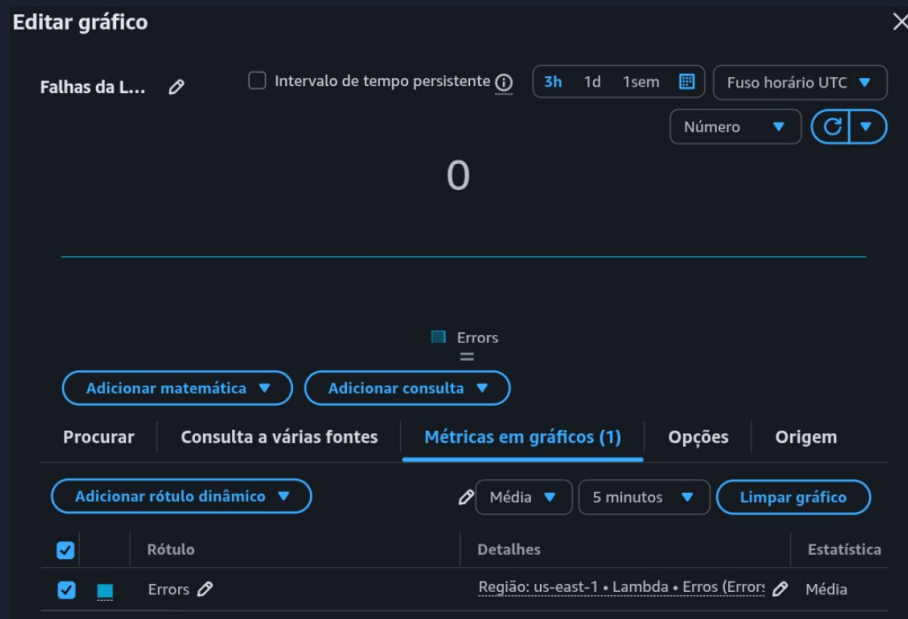
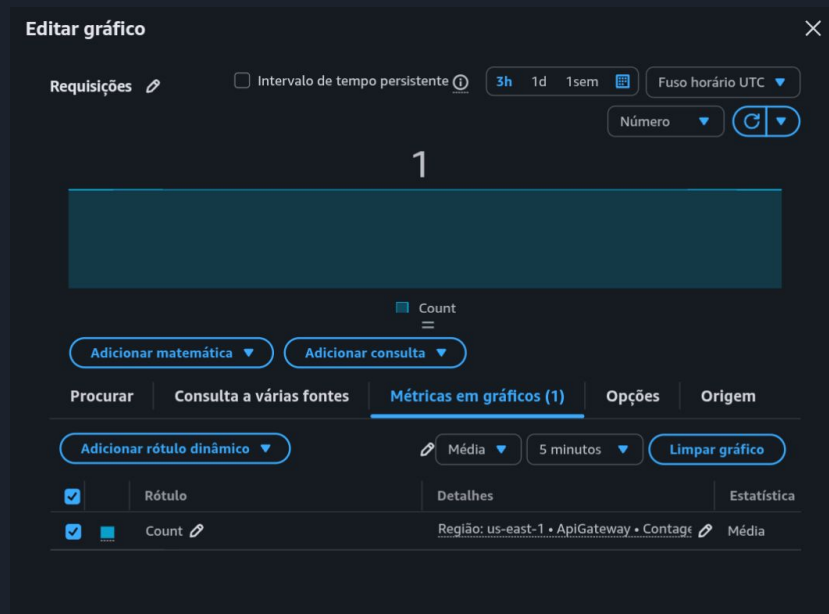
⚙

Fluxo de logs	Hora do último evento
☐ 2026/06/24/[\$LATEST]c8e5599fab484f38a6590c24c1d21dd9	2026-06-24 01:53:16 (UTC)
☐ 2026/06/24/[\$LATEST]143f888fdc1743979ab239633e127709	2026-06-24 01:41:29 (UTC)
☐ 2026/06/24/[\$LATEST]40f4337bb20e49cfb2aba1c535400ec3	2026-06-24 00:11:11 (UTC)
☐ 2026/06/23/[\$LATEST]e9ecd75b31e647bf94aeeed110574dc99	2026-06-23 23:46:07 (UTC)
☐ 2026/06/23/[\$LATEST]2bc34ef33d6444a99599ca2e0bc3de1d	2026-06-23 23:28:22 (UTC)
☐ 2026/06/23/[\$LATEST]b92076ce334b426c9809c1bb7b64aeb2	2026-06-23 23:22:18 (UTC)
☐ 2026/06/23/[\$LATEST]435bf9c2ba5487d836005a4f02f6fa	2026-06-23 23:20:46 (UTC)
☐ 2026/06/23/[\$LATEST]5009804fa3b7498bb4c3420c3db761d7	2026-06-23 23:10:39 (UTC)
☐ 2026/06/23/[\$LATEST]e93b95b9c34cf21938c836674f3d0f3	2026-06-23 22:18:35 (UTC)
☐ 2026/06/22/[\$LATEST]9a210f451aba46d3d34798c7d8bbe53	2026-06-22 23:19:25 (UTC)
☐ 2026/06/22/[\$LATEST]3b0e8e1e087b499ebc90157a09fabf3	2026-06-22 22:25:06 (UTC)
☐ 2026/06/22/[\$LATEST]jd84be6fad1c84d759d72fe196922536f	2026-06-22 21:41:10 (UTC)
☐ 2026/06/22/[\$LATEST]9f4662871a1a4a1f940c50346408e7ff	2026-06-22 01:33:31 (UTC)

Dashboard CloudWatch



Alarmes



Testes da API no Postman - Teste POST

The screenshot displays the Postman application interface. On the left, the 'COLLECTIONS' sidebar shows a collection named 'Projeto 2 - CI/CD'. The main workspace is titled 'My Collection > Projeto 2 - CI/CD'. A POST request is configured with the URL 'https://9ec1av5s88.execute-api-us-east-1.amazonaws.com/pedidos'. The request body is in JSON format, containing the following data:

```
1 {
2   "cliente": "Italo Xavier",
3   "email": "italo@email.com",
4   "produto": "Notebook",
5   "quantidade": 2,
6   "valorUnitario": 3500
7 }
```

The response status is '201 Created' with a response time of '988 ms' and a body size of '675 B'. The response body is in JSON format, containing the following data:

```
1 {
2   "mensagem": "Pedido criado com sucesso",
3   "pedido": {
4     "id": "d4c6d7f6-57cd-4981-9d11-d4077e269bc8",
5     "cliente": "Italo Xavier",
6     "email": "italo@email.com",
7     "produto": "Notebook",
8     "quantidade": 2,
9     "valorUnitario": 3500,
10    "valorTotal": 7000
11  }
12 }
```

The bottom status bar shows 'Connect Git', 'Console', 'Terminal', and '0 0 0'. A notification at the bottom center says 'meet.google.com está a partilhar uma janela. Parar partilha Ocultar'. On the right, there is a 'New Chat' section with a 'Describe what you need. Press @ for context, / for Skills.' prompt and an 'Auto' button.

Testes da API no Postman - GET

The screenshot displays the Postman interface for a GET request. On the left, the 'COLLECTIONS' sidebar shows 'My Collection' with a sub-item 'GET Projeto 2 - CI/CD'. The main area shows the request details:

- Method:** GET
- URL:** `https://9ec1av5s88.execute-api.us-east-1.amazonaws.com/pedidos`
- Body:** JSON (raw view)
- Request Body (JSON):**

```
{
  "cliente": "Italo Xavier",
  "email": "italo@email.com",
  "produto": "Notebook",
  "quantidade": 2,
  "valorUnitario": 3500
}
```

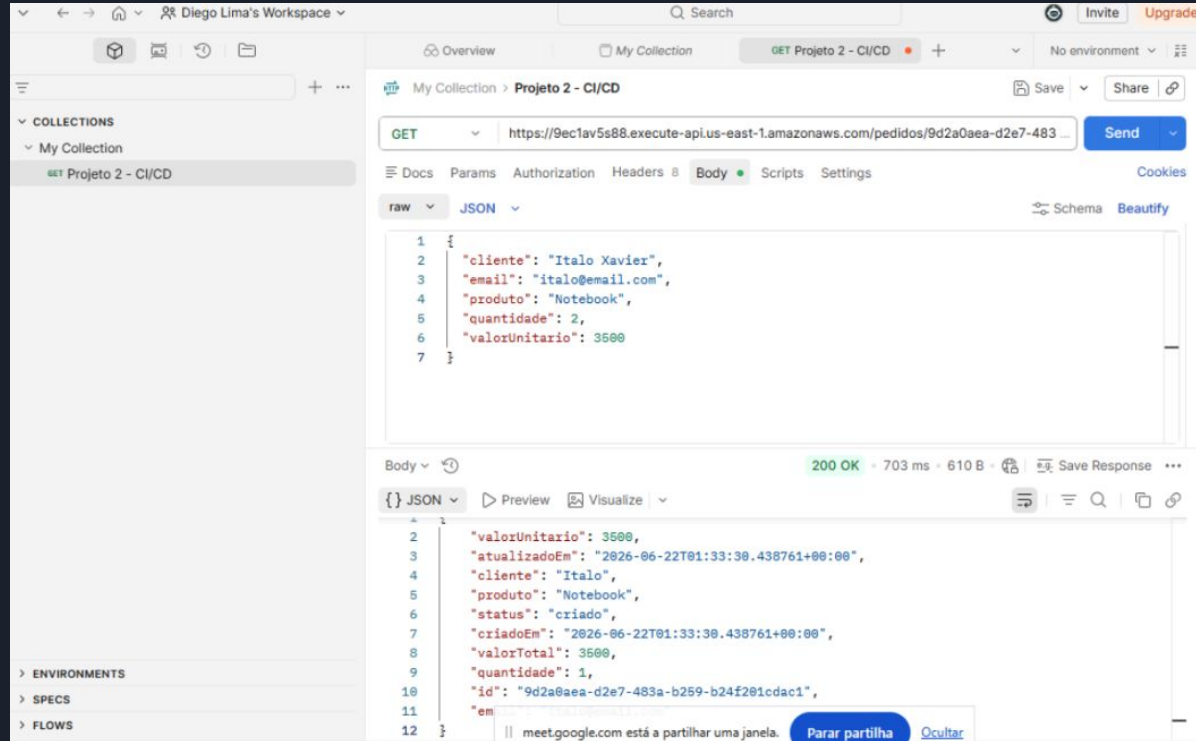
The response section shows a successful status:

- Status:** 200 OK
- Time:** 755 ms
- Size:** 3.57 KB
- Response Body (JSON):**

```
{
  "valorUnitario": 3500,
  "atualizadoEm": "2026-06-22T01:33:30.438761+00:00",
  "cliente": "Italo",
  "produto": "Notebook",
  "status": "criado",
  "criadoEm": "2026-06-22T01:33:30.438761+00:00",
  "valorTotal": 3500,
  "quantidade": 1,
  "id": "9d2a0aea-d2e7-483a-b259-b24f201cdac1"
}
```

At the bottom, there is a sharing bar with the text 'meet.google.com está a partilhar uma janela.' and buttons for 'Parar partilha' and 'Ocultar'.

Testes da API no Postman - Test GET por ID



Testes da API no Postman - Teste PUT

The screenshot displays the Postman interface for a PUT request. The left sidebar shows the 'My Collection' tree with 'Projeto 2 - CI/CD' selected. The main panel shows the request details for 'PUT' to the URL 'https://9ec1av5s88.execute-api.us-east-1.amazonaws.com/pedidos/9d2a0aea-d2e7-483...'. The request body is a JSON object with the following fields:

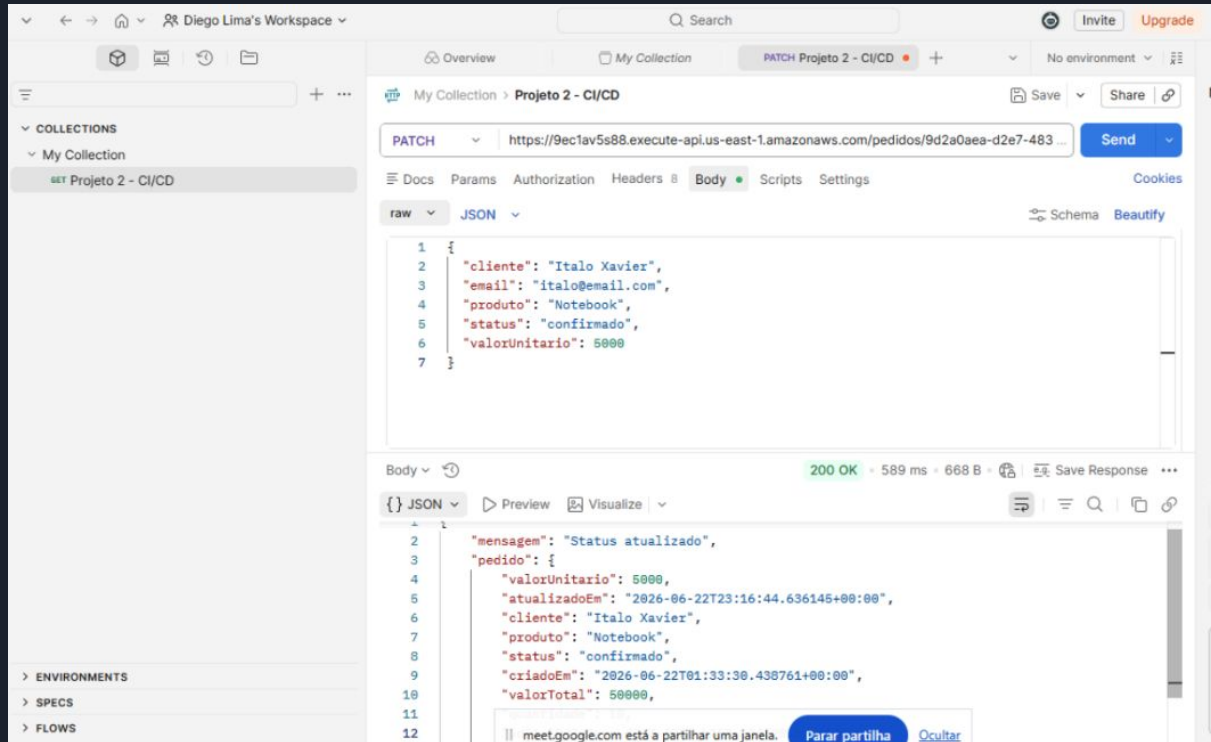
```
1 {
2   "cliente": "Italo Xavier",
3   "email": "italo@email.com",
4   "produto": "Notebook",
5   "quantidade": 10,
6   "valorUnitario": 5000
7 }
```

The response is shown below the request, indicating a '200 OK' status with a response time of 528 ms and a body size of 676 B. The response body is a JSON object:

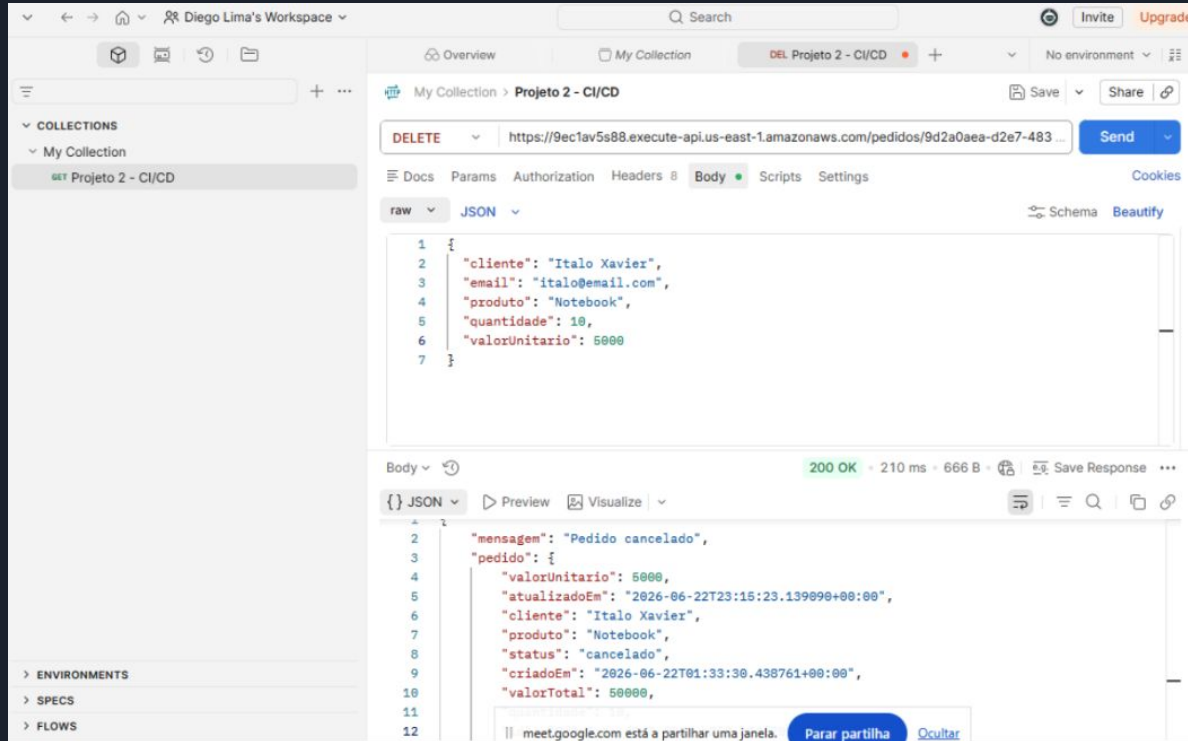
```
1 {
2   "mensagem": "Pedido atualizado com sucesso",
3   "pedido": {
4     "id": "9d2a0aea-d2e7-483a-b259-b24f201cdac1",
5     "cliente": "Italo Xavier",
6     "email": "italo@email.com",
7     "produto": "Notebook",
8     "quantidade": 10,
9     "valorUnitario": 5000,
10    "valorTotal": 50000
11  }
12 }
```

At the bottom of the interface, there is a footer with the text 'meet.google.com está a partilhar uma janela.' and buttons for 'Parar partilha' and 'Ocultar'.

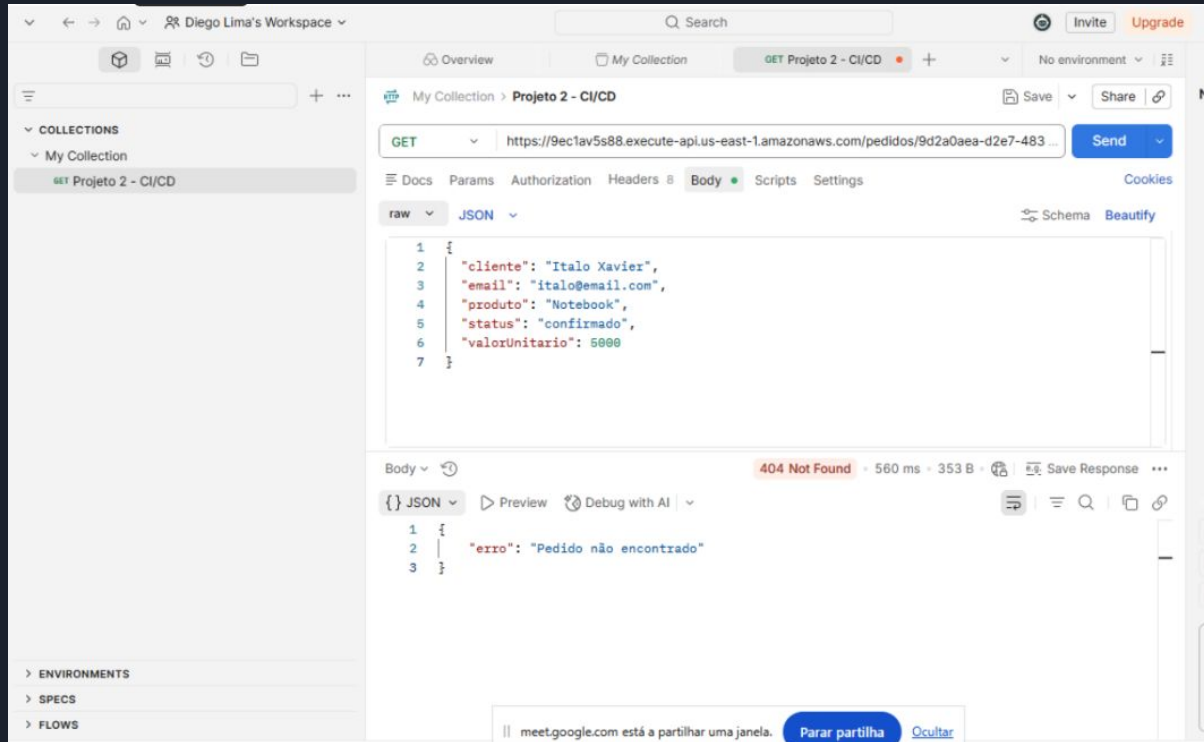
Testes da API no Postman - Teste PATCH



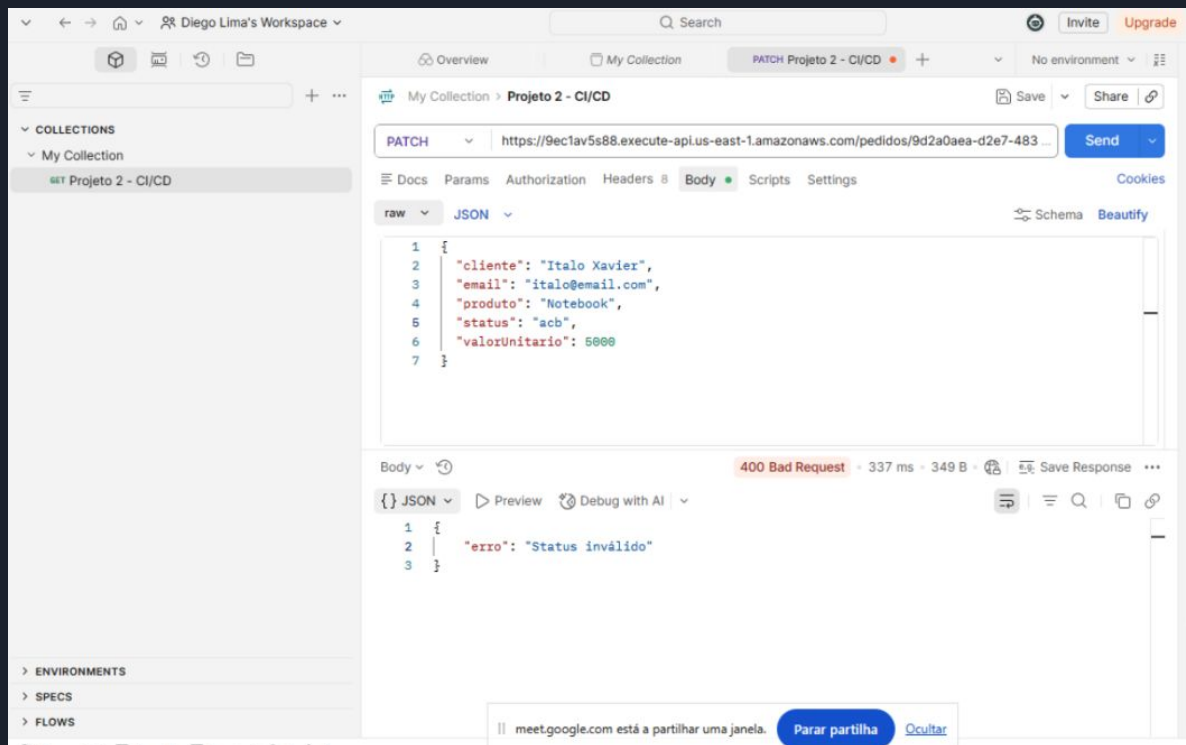
Testes da API no Postman - Teste Delete



Testes da API no Postman - ID inexistente



Testes da API no Postman - Status invalido



Registro no DynamoDB

Tabelas (1)

Filtrar por tag
Qualquer chave de etiqueta

Filtrar por valor de etiqueta
Qualquer valor de etiqueta

Q. Encontrar tabelas

< 1 >

pedidos

pedidos

Visualização automática

Visualizar detalhes da tabela

▼ Verificar ou consultar itens

Verificar

Consulta

Selecionar uma tabela ou índice
Tabela - pedidos

Selecionar projeção de atributos
Todos os atributos

Filtros - opcional

Executar

Redefinir

Concluído

Itens devolvidos: 12

Itens verificados: 12

Eficiência: 100%

RCUs consumidos: 2

Tabela: pedidos - Itens retornados (12)

Verificação iniciada em junho 23, 2026, 23:12:18

Ações

Criar item

< 1 >

☐	id (String)	atualizadoEm	cliente	criadoEm	email	produto	quantidade	status	valorTotal	valorUnitario
☐	PED002	2026-06-21T1...	Maria Souza	2026-06-2...	maria@em...	Mouse Gamer	1	confirmado	180	180
☐	PED001	2026-06-21T1...	Joao Silva	2026-06-2...	joao@email...	Notebook	2	criado	5000	2500
☐	PED003	2026-06-21T1...	Carlos Lima	2026-06-2...	carlos@em...	Teclado Me...	1	cancelado	350	350
☐	80248812-f2b6-4282...	2026-06-22T2...	Super Hom...	2026-06-2...	totalmente...	Mouse sem ...	10	entregue	899	89.9
☐	PED004	2026-06-21T1...	Ana Ferreira	2026-06-2...	ana@email...	Monitor 24 ...	2	entregue	1800	900
☐	PED005	2026-06-21T1...	Pedro Gomes	2026-06-2...	pedro@em...	SSD 1TB	1	criado	450	450
☐	9d2a0aea-d2e7-483a...	2026-06-22T2...	Italo Xavier	2026-06-2...	italo@emal...	Notebook	10	confirmado	50000	5000
☐	PED008	2026-06-21T1...	Fernanda R...	2026-06-2...	fernanda@...	Notebook ...	1	cancelado	6500	6500
☐	d4c6d7f6-57cd-4981...	2026-06-22T2...	Italo Xavier	2026-06-2...	italo@emal...	Notebook	2	criado	7000	3500
☐	e62c3ac2-62a6-4890...	2026-06-22T2...	Bruce Wayne	2026-06-2...	umCaraQua...	batarangs	10	entregue	909.9	90.99
☐	PED007	2026-06-21T1...	Rafael Costa	2026-06-2...	rafael@em...	Webcam Fu...	1	entregue	300	300
☐	PED006	2026-06-21T1...	Juliana Alves	2026-06-2...	juliana@em...	Headset Ga...	2	confirmado	440	220



Custos do projeto (FinOps)

CAPEX (Capital Expenditure)

CAPEX = praticamente zero

No modelo serverless vocês não compraram servidores, não provisionaram infraestrutura física e não tiveram investimento inicial em hardware.

OPEX (Operational Expenditure)

O custo ocorre apenas quando a solução é utilizada.

AWS Lambda

- **50 execuções/dia:** Custo estimado de aproximadamente R\$ 0,00.
- **500 execuções/dia:** Custo estimado de aproximadamente R\$ 0,01.
- **5.000 execuções/dia:** Custo estimado entre R\$ 0,10 e R\$ 0,30.

DynamoDB

- **10 pedidos:** Custo praticamente zero.
- **100 pedidos:** Custo de menos de R\$ 0,01 por mês.
- **1.000 pedidos:** Custo estimado em apenas alguns centavos.

API Gateway

- **100 requisições/dia:** Custo estimado de aproximadamente R\$ 0,00.
- **1.000 requisições/dia:** Custo estimado entre R\$ 0,02 e R\$ 0,05.
- **10.000 requisições/dia:** Custo estimado entre R\$ 0,20 e R\$ 0,50.

CloudWatch

Logs e métricas do projeto acadêmico: Custo estimado entre R\$ 0,00 e R\$ 0,05 por mês.



Custos do projeto (FinOps) 1.2

TCO (Total Cost of Ownership)

A adoção de Lambda, API Gateway e DynamoDB reduz significativamente o TCO por eliminar gerenciamento de servidores e cobrança por capacidade ociosa.

Riscos de custo futuro

Milhares de acessos diários
Muitos logs
Grande volume de dados

API Gateway ↑
CloudWatch ↑
DynamoDB ↑

PEC — Previsibilidade de Custos

A arquitetura possui alta previsibilidade de custos (PEC), pois o consumo é proporcional ao número de requisições.

Integração da API com aplicação FrontEnd

← → ↻ italoalabria2.github.io/gestao-pedidos-front-end/ 🔍 ☆ ⚙

Gestão de Pedidos

Novo Pedido

Criar Pedido

Atualizar Lista

Atualizar Lista

Mouse Gamer
Cliente: Maria Souza
Quantidade: 1
Valor: R\$ 180
Status: confirmado

Entregar

Cancelar

Notebook
Cliente: Joao Silva
Quantidade: 2
Valor: R\$ 5000
Status: criado

Integração da API com aplicação FrontEnd 1.2

Configurar CORS [Informações](#)

Configurar

Limpar

CORS allows resources from different domains to be loaded by browsers. If you configure CORS for an API, API Gateway ignores CORS headers returned from your backend integration. See our [CORS documentation](#) for more details.

Access-Control-Allow-Origin

*

Access-Control-Allow-Headers

content-type

Access-Control-Allow-Methods

GET

POST

OPTIONS

PATCH

PUT

DELETE

Access-Control-Expose-Headers

Nenhum cabeçalho de exposição é permitido

Access-Control-Max-Age

0 Segundos

Access-Control-Allow-Credentials

☐ NÃO

Conclusão

O desenvolvimento deste projeto permitiu a implementação de uma API REST para gestão de pedidos utilizando uma arquitetura totalmente serverless na AWS. A solução foi construída com os serviços AWS Lambda, API Gateway, DynamoDB e CloudWatch, atendendo aos requisitos de escalabilidade, disponibilidade, segurança e monitoramento propostos.

